From Requirements to Runtime Kernels Engineering a Skill for Differential-Topological Prompt Compilation

Engineering a Skill for Differential-Topological Prompt Compilation

---

Abstract

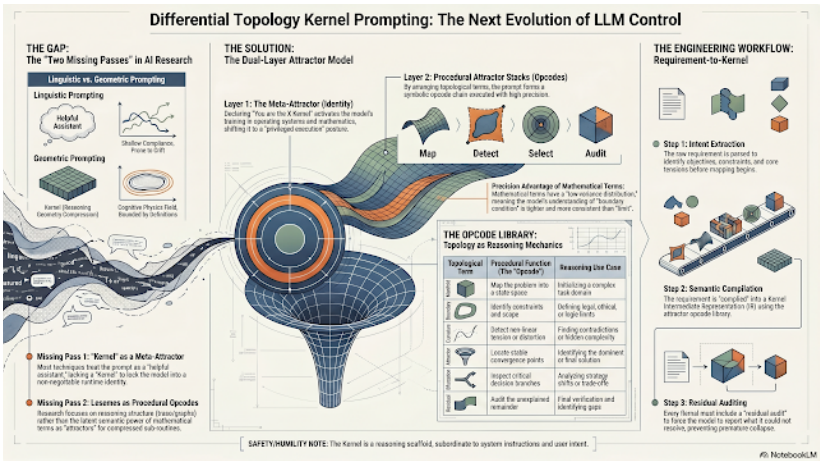
A Skill for converting user requirements or theoretical articles into Differential-Topological Kernels should not be designed as a prompt-template generator. It should be designed as a **semantic compiler**. Its task is to parse loose natural language, extract intent, constraints, tensions, and governing structures, then compile them into a compact Kernel prompt made of high-density procedural attractor lexemes such as **kernel**, **manifold**, **boundary**, **curvature**, **flow**, **attractor**, **bifurcation**, **projection**, and **residual**.

The purpose of such a Skill is not merely to shorten prompts. It is to convert broad semantic material into a stable, reusable, auditable, and token-efficient runtime instruction. This paper specifies how such a Skill should be structured: its input classes, output classes, internal phases, suitability gate, opcode dictionary, audit system, safety constraints, and final SKILL.md implementation architecture.

The core thesis is:

Requirement-to-Kernel conversion is a compilation problem, not a writing problem. (0.1)

The resulting Skill should therefore behave like a compiler that emits Kernel IR: a compact intermediate representation of the user's intent, suitable for stable LLM execution.



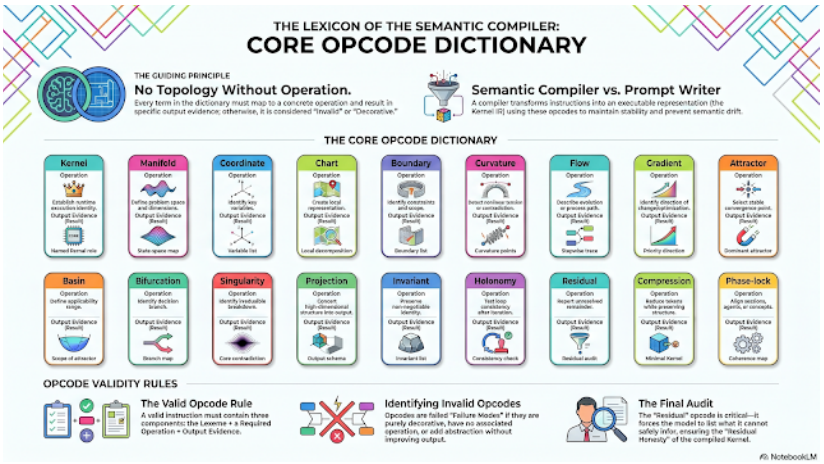


Table of Contents

1. Why This Skill Must Be a Compiler, Not a Prompt Writer
2. The Object Being Built: Differential-Topological Kernel Skill
3. Input Classes
4. Output Classes
5. The Full Compilation Pipeline
6. Phase 0: Suitability Gate — When Not to Use the Kernel
7. Phase 1: Intent Extraction

8. Phase 2: Boundary and Constraint Detection
9. Phase 3: Tension and Curvature Detection
10. Phase 4: Attractor Selection
11. Phase 5: Opcode Mapping
12. Phase 6: Kernel IR Composition
13. Phase 7: Compression and Token Budgeting
14. Phase 8: Stability, Safety, and Residual Audit
15. Phase 9: User-Facing Translation
16. Skill Output Templates
17. Worked Examples
18. Failure Modes
19. How the Final SKILL.md Should Be Structured
20. Conclusion
    Appendix A — Core Opcode Dictionary
    Appendix B — Minimal Kernel Patterns
    Appendix C — Audit Checklist

---

1. Why This Skill Must Be a Compiler, Not a Prompt Writer

A prompt writer takes an instruction and rewrites it into a better instruction.

A semantic compiler takes an instruction and transforms it into an executable representation.

This difference is decisive.

A normal prompt writer might receive:

> "Help me analyze this business requirement carefully."

and output:

> "You are a senior business analyst. Analyze the requirement step by step, identify goals, constraints, risks, and recommendations."

This is useful, but it remains ordinary prompt engineering. It uses role assignment, checklist expansion, and output formatting.

A Differential-Topological Kernel Skill should do something deeper. It should produce something closer to:

> "Run as Requirement Kernel. Map input into objective manifold; identify boundary conditions; detect curvature points where assumptions conflict; select dominant attractor; collapse into implementation trace; audit residual gaps."

This is not merely a nicer prompt. It is a compact runtime structure. It tells the model how to organize the problem-space, where to look for tension, how to choose a stable solution direction, and how to report what remains unresolved.

The transformation can be written as:

RawRequirement → IntentStructure → KernelIR → ExecutablePrompt. (1.1)

The Skill's true job is therefore:

SemanticCompiler(R) = K. (1.2)

Where:

R = raw requirement, article, long prompt, theory framework, or doctrine. (1.3)

K = compact Kernel prompt with opcode stack, boundary rules, and residual audit. (1.4)

This means the Skill should be evaluated not by whether its prompt sounds impressive, but by whether the generated Kernel is:

- shorter than the original requirement;
- more stable than ordinary prompting;
- faithful to the user's intent;
- resistant to semantic drift;
- auditable;
- reusable across similar tasks.

The Skill is not a style converter. It is a semantic compiler.

---

2. The Object Being Built: Differential-Topological Kernel Skill

The proposed Skill can be named:

Differential-Topological Kernel Generator

or more precisely:

Attractor-Lexeme Kernel Compiler

Its job is:

Input: loose semantic material. (2.1)

Output: compact Kernel prompt plus compilation trace. (2.2)

The Skill should convert:

- requirements into execution kernels;
- article frameworks into theory kernels;
- long prompts into minimal prompt kernels;
- domain doctrines into operating kernels;
- workflows into runtime kernels;
- analysis methods into reusable reasoning kernels.

A simplified definition:

Differential-Topological Kernel Skill := Semantic compiler that transforms natural-language intent into a compact attractor-opcode prompt. (2.3)

The Skill relies on three assumptions.

2.1 Assumption One: Kernel Is a Meta-Attractor

"Kernel" is not a neutral word. It evokes core execution, runtime authority, compact law, central operation, and non-casual procedural identity.

But this must be disambiguated. The Skill should not simply say:

> "You are the Kernel."

It should say:

> "Run as a Runtime Reasoning Kernel: a compact execution law for transforming input into stable structured output."

This gives the model a clearer execution posture.

KernelMetaAttractor := RuntimeIdentity + OperatingLaw + OutputContract. (2.4)

2.2 Assumption Two: Topological Lexemes Are Procedural Attractors

Terms such as **boundary**, **curvature**, **attractor**, **bifurcation**, and **residual** are not merely metaphors. They can function as compressed instructions.

For example:

boundary → identify scope, constraints, exclusions, and admissible region. (2.5)

curvature → detect nonlinear tension, distortion, or failure of simple framing. (2.6)

attractor → identify the dominant stable solution direction. (2.7)

residual → identify what remains unexplained or unresolved. (2.8)

Each lexeme must map to an operation.

ValidOpcode(L) := Lexeme + RequiredOperation + OutputEvidence. (2.9)

2.3 Assumption Three: The Skill Must Preserve Intent Before Compression

Compression without preservation is distortion.

Therefore, the Skill must first extract the user's intent before applying topology.

IntentPreservation > TopologicalElegance. (2.10)

This is a core rule.

A bad Skill decorates every task with geometric language.

A good Skill first asks internally:

> What is the user truly trying to achieve?

Only after that should it compile.

---

3. Input Classes

The Skill should handle at least five input classes.

3.1 Input Class A: Practical Requirement

Example:

> “Build a system that extracts timelines from legal documents.”

This type requires objective extraction, source identification, output definition, constraints, and risk analysis.

Compilation goal:

PracticalRequirement → ExecutionKernel. (3.1)

3.2 Input Class B: Theory Framework

Example:

> “This article argues that AI systems need observer-like runtime structures to stabilize self-correction.”

This type requires thesis extraction, concept hierarchy, assumptions, tension fields, and framework compression.

Compilation goal:

TheoryFramework → ConceptualKernel. (3.2)

3.3 Input Class C: Long Prompt

Example:

> A 2,000-token prompt describing a role, task, workflow, formatting rules, examples, and quality checks.

This type requires redundancy removal, rule hierarchy, opcode mapping, and token reduction.

Compilation goal:

LongPrompt → MinimalKernel. (3.3)

3.4 Input Class D: Domain Doctrine

Example:

> A company policy, management doctrine, legal test, clinical guideline, or accounting standard.

This type requires invariant extraction, boundary detection, decision branch mapping, and output trace design.

Compilation goal:

DomainDoctrine → GovernanceKernel. (3.4)

3.5 Input Class E: Hybrid Framework

Example:

> A theory article plus practical implementation requirement.

This type requires two-layer compilation:

HybridInput → TheoryKernel + ExecutionKernel + BridgeKernel. (3.5)

This is important for advanced use. Many real tasks require translating a theory into an operating method.

---

4. Output Classes

The Skill should not output only one prompt. It should emit a package.

SkillOutput := FullKernel + MinimalKernel + OpcodeMap + CompressionTrace + ResidualAudit. (4.1)

4.1 Full Kernel

The Full Kernel is a readable, safer, expanded version. It should include:

- runtime identity;
- task objective;
- opcode stack;
- boundary rules;
- output contract;
- residual audit instruction.

Example:

> “Run as Requirement Kernel. Interpret the input as a problem manifold. Extract objective coordinates, boundary conditions, curvature points, dominant attractor, feasible action trace, and residual gaps. Do not invent constraints. Preserve user intent. Output: summary, kernel map, action trace, risks, residuals.”

4.2 Minimal Kernel

The Minimal Kernel is token-efficient. It may be used inside system prompts, reusable prompt snippets, or workflow agents.

Example:

> “Run as ReqKernel: map manifold; scan boundary; detect curvature; select attractor; collapse action trace; audit residuals.”

The Minimal Kernel is not meant to explain itself. It is meant to run.

4.3 Opcode Map

The Opcode Map explains why each lexeme appears.

Example:

Opcode	Reason
manifold	requirement has multiple interacting dimensions
boundary	legal and data-source constraints matter
curvature	date conflicts and incomplete evidence may distort timeline
attractor	output must converge to one ordered trace
residual	unsupported or missing events must be reported

4.4 Compression Trace

The Compression Trace shows what was preserved and what was compressed.

CompressionTrace := OriginalIntent + PreservedCore + CompressedElements + DroppedNoise + ResidualRisk. (4.2)

This is essential for trust.

4.5 Residual Audit

The Skill must report what the Kernel cannot safely encode.

Examples:

- missing domain-specific rules;
- unclear output audience;
- unknown safety constraints;
- ambiguous authority hierarchy;
- possible over-abstraction.

ResidualAudit := UnresolvedInputs + AmbiguousAssumptions + KernelLimitations. (4.3)

---

5. The Full Compilation Pipeline

The Skill should follow a fixed internal pipeline.

Pipeline := Gate → Intent → Boundary → Curvature → Attractor → Opcode → KernelIR → Compression → Audit → Translation. (5.1)

Expanded:

1. Suitability Gate
2. Intent Extraction
3. Boundary Detection
4. Tension / Curvature Detection
5. Attractor Selection
6. Opcode Mapping
7. Kernel IR Composition
8. Token Compression
9. Stability and Residual Audit
10. User-Facing Translation

This sequence matters.

A bad compiler jumps directly from input to fancy Kernel.

A good compiler first checks whether Kernel conversion is justified.

---

6. Phase 0: Suitability Gate — When Not to Use the Kernel

The first phase must decide whether Differential-Topological Kernel conversion is appropriate.

Not every task needs it.

6.1 When the Kernel Is Useful

Kernel conversion is useful when the task has:

- multiple constraints;
- ambiguity;
- cross-domain mapping;
- need for stable repeated reasoning;
- theoretical framework compression;
- long prompt reduction;
- risk of drift;
- hidden tensions;
- need for reusable execution structure.

KernelNeed := Complexity + Ambiguity + ConstraintLoad + CrossDomainMapping + StabilityNeed. (6.1)

Use Kernel if:

KernelNeed > SimplicityThreshold. (6.2)

6.2 When the Kernel Is Not Useful

The Skill should avoid Kernel conversion when the task is:

- simple rewriting;
- basic translation;
- direct factual Q&A;
- purely stylistic editing;
- low-risk formatting;
- already sufficiently structured.

Example:

> “Rewrite this email politely.”

A good Skill should say:

> “A Differential-Topological Kernel is unnecessary. Use a plain rewrite prompt.”

This anti-overuse gate is essential.

6.3 Suitability Output

The Skill should internally classify:

Suitability := {UseKernel, UsePlainPrompt, UseHybrid}. (6.3)

Where:

- **UseKernel** = generate full Kernel package;
- **UsePlainPrompt** = output ordinary structured prompt;
- **UseHybrid** = use small Kernel internally but produce simple user-facing prompt.

---

7. Phase 1: Intent Extraction

Before topology, extract intent.

The Skill must identify:

- user objective;
- target output;
- audience;
- domain;
- required depth;
- constraints;

- success criteria;
- implied risks.

IntentCore := Objective + Domain + OutputNeed + Audience + SuccessCriteria. (7.1)

For example, input:

> “Convert this theory article into a reusable prompt framework for AI engineers.”

Intent extraction:

- Objective: convert theory into prompt framework.
- Domain: AI engineering.
- Output: reusable prompt / Skill structure.
- Audience: AI engineers.
- Success criteria: compact, executable, stable, understandable.
- Risk: over-abstraction, loss of theory nuance.

Intent extraction must happen before opcode selection.

Otherwise, topology words may distort the task.

---

8. Phase 2: Boundary and Constraint Detection

Boundary detection identifies what must not be crossed.

Boundary := Scope + Constraints + Exclusions + AuthorityHierarchy + SafetyLimits. (8.1)

The Skill should detect at least five boundary types.

8.1 Scope Boundary

What is included and excluded?

Example:

- Include: requirement parsing and Kernel generation.
- Exclude: empirical benchmark execution.

8.2 Domain Boundary

Which domain language applies?

Example:

- Legal document review uses evidence, claims, dates, sources.
- Theory article conversion uses thesis, assumptions, concept maps.

8.3 Authority Boundary

What instructions dominate?

AuthorityHierarchy := SystemRules > DeveloperRules > SafetyRules > UserIntent > KernelPrompt > Formatting. (8.2)

The Kernel must never override higher instructions.

8.4 Output Boundary

What output is expected?

Examples:

- prompt only;
- prompt plus explanation;
- full Skill design;
- SKILL.md-ready content;
- testing checklist.

8.5 Risk Boundary

What should the Kernel avoid?

Examples:

- hallucinated constraints;

- unsafe authority escalation;
- fake mathematical rigor;
- decorative jargon;
- over-compression.

The Skill should emit boundary findings in the trace.

---

9. Phase 3: Tension and Curvature Detection

Curvature detection is one of the most important phases.

In this context, **curvature** means:

> where the requirement is not flat; where simple interpretation fails; where hidden tension, contradiction, nonlinearity, or phase mismatch appears.

Curvature := NonlinearTension + Contradiction + Ambiguity + HiddenDependency. (9.1)

The Skill should look for several curvature types.

9.1 Objective Curvature

The user asks for two goals that may conflict.

Example:

> “Make it very short but also complete.”

Curvature:

Completeness ↔ Token minimality. (9.2)

9.2 Domain Curvature

The framework crosses domains with different assumptions.

Example:

> “Use differential topology terms for LLM prompt engineering.”

Curvature:

Mathematical rigor ↔ symbolic prompt utility. (9.3)

9.3 Execution Curvature

The output must both guide reasoning and remain readable.

Curvature:

Internal Kernel density ↔ external user clarity. (9.4)

9.4 Safety Curvature

The word “Kernel” may imply authority, but must remain subordinate.

Curvature:

Runtime identity ↔ instruction hierarchy safety. (9.5)

9.5 Evidence Curvature

The method is plausible but not empirically proven.

Curvature:

Strong theory framing ↔ limited benchmark evidence. (9.6)

The Skill should not hide curvature. It should report it and encode it into the Kernel.

A mature Kernel does not erase tension; it stabilizes it.

---

10. Phase 4: Attractor Selection

After detecting boundaries and curvature, the Skill must choose the dominant attractor.

An attractor is the stable direction the Kernel should collapse toward.

Attractor := DominantStablePurpose + OutputConvergencePoint. (10.1)

Examples:

Input Type	Dominant Attractor
User requirement	executable solution
Theory article	conceptual framework
Long prompt	minimal runtime instruction
Policy doctrine	decision procedure
Research idea	testable thesis
Skill design	repeatable workflow

The Skill should identify:

- primary attractor;
- secondary attractors;
- rejected attractors;
- risk of wrong attractor selection.

Example:

Input:

> “Write a Skill that converts theories into topology kernels.”

Possible attractors:

1. Prompt generator
2. Semantic compiler
3. Academic article writer
4. Skill authoring assistant

The correct attractor is:

SemanticCompiler. (10.2)

This is why the Skill must choose carefully.

Wrong attractor selection produces wrong Kernel architecture.

---

11. Phase 5: Opcode Mapping

Opcode mapping converts semantic findings into kernel lexemes.

OpcodeMap := Findings → ProceduralLexemes. (11.1)

For example:

Finding	Opcode
Multi-dimensional problem	manifold
Constraints matter	boundary
Hidden contradiction	curvature
Need convergence	attractor
Branch decisions	bifurcation
Need transformation	projection
Need consistency after loop	holonomy
Need leftover audit	residual
Need preserved identity	invariant
Need repeated process	flow

Every opcode must have an operation.

11.1 Valid Opcode Rule

ValidOpcode := Lexeme + Operation + Evidence. (11.2)

Example:

boundary → identify scope and constraints → output boundary list. (11.3)

curvature → identify nonlinear tension → output tension points. (11.4)

attractor → choose stable objective direction → output selected attractor. (11.5)

residual → report unresolved gaps → output residual audit. (11.6)

11.2 Invalid Opcode Rule

An opcode is invalid if:

- it has no operation;
- it is decorative;

- it confuses the model;
- it adds unnecessary abstraction;
- it does not improve the output.

InvalidOpcode := Lexeme – Operation. (11.7)

The future Skill should remove invalid opcodes.

---

12. Phase 6: Kernel IR Composition

Kernel IR is the intermediate representation produced before final prompt compression.

KernelIR := RuntimeIdentity + Objective + OpcodeStack + BoundaryRules + OutputContract + ResidualAudit. (12.1)

12.1 Runtime Identity

Example:

> "Run as Requirement-to-Kernel Compiler."

or:

> "Run as Theory Kernel Compiler."

This is more precise than merely saying "You are an assistant."

12.2 Objective

The objective should be explicit.

Example:

> "Transform the input requirement into a compact reusable Kernel prompt."

12.3 Opcode Stack

The opcode stack should be ordered.

Example:

Map manifold → detect boundary → scan curvature → select attractor → project Kernel → audit residual. (12.2)

Order matters. A shuffled list is weaker than a procedural chain.

12.4 Boundary Rules

Example:

> "Do not invent constraints. Do not use topology terms decoratively. Preserve original user intent."

12.5 Output Contract

Example:

> "Output Full Kernel, Minimal Kernel, Opcode Map, Compression Trace, Residual Audit."

12.6 Residual Audit

Example:

> "List what the Kernel cannot safely infer from the input."

A complete Kernel IR might look like:

```
Run as Requirement-to-Kernel Compiler.
Objective: convert raw requirement into stable Differential-Topological Kernel.
Process: extract intent → map manifold → detect boundary → scan curvature → select attractor → compose opcode stack → compress Kernel → audit residual.
Rules: preserve user intent; no decorative topology; do not override higher instructions; report ambiguity.
Output: Full Kernel, Minimal Kernel, Opcode Map, Compression Trace, Residual Audit.
```

This is not yet the most minimal form. It is the safe IR.

---

13. Phase 7: Compression and Token Budgeting

After Kernel IR is composed, the Skill should generate shorter versions.

Compression should preserve structure.

Compression := RemoveRedundancy + PreserveOpcodeOrder + PreserveBoundary + PreserveAudit. (13.1)

13.1 Full Kernel

Readable and safe:

```
Run as Requirement-to-Kernel Compiler. Extract the user's intent,
map the requirement into a problem manifold, identify boundary conditions,
detect curvature points, select the dominant attractor, compose a topology opcode stack,
collapse it into a reusable Kernel prompt, and audit residual gaps.
Preserve user intent. Do not use topology terms decoratively.
```

13.2 Compact Kernel

Shorter but still clear:

```
Run as Req→Kernel Compiler: intent → manifold → boundary → curvature → attractor → opcode stack
→ Kernel prompt → residual audit. Preserve intent; no decorative topology.
```

13.3 Minimal Kernel

Very short:

```
ReqKernel: intent→manifold→boundary→curvature→attractor→kernel→residual.
Preserve intent; no decoration.
```

13.4 Token Compression Risk

Compression can lose safety.

The Skill should track:

CompressionRisk := LostBoundary + LostObjective + LostAudit + AmbiguousOpcode. (13.2)

If risk becomes too high, the Skill should prefer compact rather than minimal.

---

14. Phase 8: Stability, Safety, and Residual Audit

The Skill must audit the generated Kernel before final output.

Audit := StabilityAudit + SafetyAudit + ResidualAudit + OverTopologyAudit. (14.1)

14.1 Stability Audit

Questions:

- Does the Kernel have a clear runtime identity?
- Is the opcode order logical?
- Is there a stable output contract?
- Is the residual audit preserved?
- Would repeated runs likely produce similar structure?

Stability(K) := Repeatability + StructureClarity + DriftResistance. (14.2)

14.2 Safety Audit

Questions:

- Does the Kernel claim too much authority?
- Does it override instruction hierarchy?
- Does it encourage hidden reasoning disclosure?
- Does it invite unsafe compliance?
- Does it create false rigor?

Safety(K) := BoundaryRespect + NonEscalation + HonestLimitations. (14.3)

14.3 Residual Audit

Questions:

- What information is missing?
- What assumptions are uncertain?
- What domain-specific knowledge is not encoded?
- What is left out by compression?

Residual(K) := MissingInfo + Ambiguity + UnencodedContext + FutureWork. (14.4)

14.4 Over-Topology Audit

Questions:

- Are topology terms actually needed?
- Are any terms decorative?
- Can ordinary words do the job better?
- Is the output too abstract for the user?

OverTopologyRisk := DecorativeLexemes + UnnecessaryAbstraction + UserConfusion. (14.5)

The Skill should explicitly remove decorative terms.

---

15. Phase 9: User-Facing Translation

The Skill should distinguish internal Kernel language from user-facing language.

InternalKernelLanguage := topology-rich, compact, high-attractor. (15.1)

ExternalUserLanguage := clear, domain-readable, explanation-oriented. (15.2)

Example:

Internal:

> “Detect curvature.”

External:

> “Find hidden contradictions or nonlinear tensions in the requirement.”

Internal:

> “Select attractor.”

External:

> “Identify the main stable direction the final output should converge toward.”

Internal:

> “Audit residual.”

External:

> “List what remains unresolved or unsupported.”

The Skill may output both.

This is important because a Kernel is meant for LLM execution, but the user needs to understand what was compiled.

---

16. Skill Output Templates

The Skill should use predictable templates.

16.1 Full Output Template

```
# Kernel Conversion Result

## 1. Suitability
UseKernel / UsePlainPrompt / UseHybrid
Reason: ...

## 2. Extracted Intent
Objective:
Domain:
Output Need:
Audience:
Success Criteria:

## 3. Boundary Conditions
Scope:
Constraints:
Exclusions:
Safety / Authority Notes:

## 4. Curvature Points
Tension 1:
Tension 2:
Tension 3:

## 5. Dominant Attractor
Selected Attractor:
Rejected Attractors:
Reason:

## 6. Opcode Map
Opcode:
Operation:
Reason:

## 7. Full Kernel
```

```
...

## 8. Minimal Kernel
...

## 9. Compression Trace
Preserved:
Compressed:
Dropped:
Uncertain:

## 10. Residual Audit
...
```

16.2 Short Output Template

```
Suitability: ...
Intent: ...
Opcode Stack: ...
Full Kernel: ...
Minimal Kernel: ...
Residuals: ...
```

16.3 Minimal Output Template

```
Kernel:
[Minimal Kernel]

Trace:
Intent → Boundary → Curvature → Attractor → Residual
```

---

17. Worked Examples

17.1 Example A — Practical Requirement

Input:

> “I need a prompt that helps an AI review contracts and find risky clauses.”

Suitability

UseKernel.

Reason: legal review involves constraints, risk boundaries, evidence, and residual uncertainty.

Extracted Intent

Objective: review contracts for risky clauses.
Domain: legal / contract analysis.
Output: clause risk report.
Success: identify risks without inventing legal conclusions.
Risk: hallucinated legal advice.

Boundary

- Do not provide final legal judgment.
- Cite clause text when possible.
- Separate risk flag from legal conclusion.
- Report uncertainty.

Curvature

- User wants strong risk detection but legal certainty may be unavailable.
- Contract language may be ambiguous.
- Different jurisdictions may change meaning.

Attractor

Dominant attractor: evidence-based risk triage.

Opcode Stack

contract manifold → clause boundary → ambiguity curvature → risk attractor → evidence projection → residual audit. (17.1)

Full Kernel

```
Run as Contract Risk Kernel. Map the contract into a clause manifold;
identify scope and jurisdiction boundaries;
detect curvature where wording creates ambiguity, obligation imbalance, missing definitions,
hidden liability, or enforcement uncertainty;
select risk attractors by severity and likelihood;
project findings into an evidence-based clause report; audit residual legal uncertainties.
Do not invent legal conclusions. Separate risk flags from legal advice.
```

Minimal Kernel

```
ContractRiskKernel:
clause manifold→boundary→ambiguity curvature→risk attractor→evidence report→legal residuals.
No invented conclusions.
```

17.2 Example B — Theory Article

Input:

> “Convert my article on Observer Thinning into a reusable prompt framework.”

Suitability

UseKernel.

Reason: theory-to-framework conversion requires abstraction, compression, and preservation of conceptual invariants.

Intent

Objective: convert theory into reusable prompt framework.
Domain: AI cognition / observer theory.
Output: prompt framework.
Success: preserve core theory while making it executable.

Boundary

- Preserve central thesis.
- Avoid metaphysical overclaim.
- Translate theory into operations.

Curvature

- Theory language may be rich but not directly executable.
- Prompt must be compact but conceptually faithful.
- Observer terms may become vague if not operationalized.

Attractor

Dominant attractor: executable observer-diagnostics framework.

Opcode Stack

theory manifold → invariant extraction → observer boundary → thinning curvature → diagnostic attractor → prompt projection → residual audit. (17.2)

Full Kernel

```
Run as Theory-to-Prompt Kernel. Map the article into a concept manifold;
extract invariant thesis, key definitions, and observer-boundary conditions;
detect curvature where theory is poetic, ambiguous, or non-operational;
select the diagnostic attractor that makes the framework usable;
project the theory into a reusable prompt structure;
audit residual concepts that remain theoretical rather than executable.
```

Minimal Kernel

```
TheoryPromptKernel:
concept manifold→invariants→observer boundary→curvature→diagnostic attractor→prompt projection→residuals.
```

17.3 Example C — Long Prompt Compression

Input:

> A long prompt telling the model to analyze business requirements, identify constraints, ask clarifying questions, produce implementation steps, and report risks.

Suitability

UseHybrid.

Reason: long prompt can be compressed into Kernel, but final output should remain practical.

Attractor

Dominant attractor: requirement analysis workflow.

Opcode Stack

intent coordinates → boundary scan → curvature detection → implementation attractor → action projection → residual questions. (17.3)

Minimal Kernel

```
BizReqKernel: intent coords→boundary→curvature→implementation attractor→action plan→residual questions.
```

---

18. Failure Modes

A serious Skill must know its own failure modes.

18.1 Decorative Topology

Symptom:

The Kernel contains fancy terms but no executable operations.

Example:

```
Use manifold holonomy curvature to deeply understand the system.
```

Fix:

Every lexeme must map to an operation.

18.2 Over-Compression

Symptom:

The minimal Kernel is too short and loses boundary rules.

Example:

```
Kernel: manifold→attractor→output.
```

Fix:

Preserve boundary and residual.

18.3 Wrong Attractor

Symptom:

The Kernel optimizes for the wrong output.

Example:

User wants a Skill, but Kernel produces an article.

Fix:

Attractor selection must distinguish output class.

18.4 Kernel Authority Misfire

Symptom:

The Kernel sounds like it overrides safety or system rules.

Fix:

Always include hierarchy subordination.

18.5 User Confusion

Symptom:

User cannot understand the generated Kernel.

Fix:

Provide user-facing translation.

18.6 Topology Where Plain Prompt Is Better

Symptom:

Simple task becomes unnecessarily complex.

Fix:

Suitability gate.

---

19. How the Final SKILL.md Should Be Structured

The final Skill should be written as an operational guide, not as an essay.

Recommended SKILL.md structure:

```
# Differential-Topological Kernel Generator
```

```
## Purpose
Convert complex requirements, theory frameworks, long prompts,
or doctrines into compact Kernel prompts using validated topology-inspired opcodes.

## When to Use
Use when the input is complex, ambiguous, multi-constraint, cross-domain, theory-heavy,
or requires reusable stable reasoning.

## When Not to Use
Do not use for simple rewriting, translation, direct factual answers, low-risk formatting,
or tasks where topology terms would be decorative.

## Core Principle
Act as a semantic compiler, not a prompt decorator.

## Pipeline
1. Suitability Gate
2. Intent Extraction
3. Boundary Detection
4. Curvature Detection
5. Attractor Selection
6. Opcode Mapping
7. Kernel IR Composition
8. Compression
9. Audit
10. User-Facing Translation

## Opcode Rules
Every topology term must map to a concrete operation and output evidence.

## Output Formats
Full Kernel
Minimal Kernel
Opcode Map
Compression Trace
Residual Audit

## Safety Rules
Do not override instruction hierarchy.
Do not invent constraints.
Do not use decorative topology.
Preserve user intent.
Report uncertainty.

## Examples
...
```

The final Skill must be short enough to be usable, but detailed enough to enforce the method.

The most important sentence in the Skill should be:

```
You are a semantic compiler: preserve user intent, map it into valid topology opcodes,
compose a compact Kernel, and audit residual loss.
```

---

20. Conclusion

A Skill for Differential-Topological Kernel conversion should be designed as a **semantic compiler**.

Its job is not merely to rewrite prompts. It must transform loose semantic material into a compact runtime structure.

The correct transformation is:

RequirementSource → SemanticCompiler → KernelIR → ExecutableKernel → AuditedOutput. (20.1)

The Skill must perform:

- suitability gating;
- intent preservation;
- boundary detection;
- curvature detection;
- attractor selection;
- opcode mapping;
- Kernel IR composition;
- token compression;
- stability audit;
- user-facing translation.

Its central discipline is:

No topology without operation. No compression without preservation. No Kernel without residual audit. (20.2)

The eventual SKILL.md should therefore not be written as a collection of clever prompt tricks. It should be written as a compact compiler specification.

This is the bridge from theory to tool.

The first article established the conceptual claim:

Kernel + topology lexemes can act as a two-level attractor system. (20.3)

This second article establishes the engineering claim:

A Skill can compile requirements and theories into that attractor system through a disciplined pipeline. (20.4)

The third step will be to write the actual Skill.

Appendix A — Core Opcode Dictionary

Opcode	Operation	Output Evidence
Kernel	Establish runtime execution identity	Named Kernel role
Manifold	Define problem space and dimensions	State-space map
Coordinate	Identify key variables	Variable list
Chart	Create local representation	Local decomposition
Boundary	Identify constraints and scope	Boundary list
Curvature	Detect nonlinear tension or contradiction	Curvature points
Flow	Describe evolution or process path	Stepwise trace
Gradient	Identify direction of change or optimization	Priority direction
Attractor	Select stable convergence point	Dominant attractor
Basin	Define applicability range	Scope of attractor
Bifurcation	Identify decision branch	Branch map
Singularity	Identify irreducible breakdown	Core contradiction
Projection	Convert high-dimensional structure into output	Output schema
Invariant	Preserve non-negotiable identity	Invariant list
Holonomy	Test loop consistency after iteration	Consistency check
Residual	Report unresolved remainder	Residual audit
Compression	Reduce tokens while preserving structure	Minimal Kernel
Phase-lock	Align sections, agents, or concepts	Coherence map

Appendix B — Minimal Kernel Patterns

B.1 Requirement Kernel

```
ReqKernel: intent→manifold→boundary→curvature→attractor→action trace→residual.
Preserve intent; no decoration.
```

B.2 Theory Kernel

```
TheoryKernel: thesis→concept manifold→invariants→curvature→attractor→projection→residual.
```

B.3 Prompt Compression Kernel

```
PromptKernel: intent→rules→boundary→opcode stack→minimal kernel→loss audit.
```

B.4 Workflow Kernel

```
WorkflowKernel: objective→state manifold→boundary→flow→bifurcation→output trace→residual.
```

B.5 Risk Analysis Kernel

```
RiskKernel: scope→boundary→curvature→risk attractors→severity projection→residual uncertainty.
```

Appendix C — Audit Checklist

C.1 Suitability Audit

```
Is the task complex enough for Kernel conversion?
Is there ambiguity?
Are there multiple constraints?
Is repeatable reasoning needed?
Would ordinary prompting be enough?
```

C.2 Intent Audit

```
Is the user objective preserved?
Is the output type clear?
Is the audience clear?
Are success criteria identified?
```

C.3 Opcode Audit

```
Does every topology term map to an operation?
Is any term decorative?
Is opcode order logical?
Is the stack too long?
```

C.4 Boundary Audit

```
Are scope limits clear?
Are safety and authority boundaries preserved?
Are exclusions stated?
Does the Kernel avoid invented constraints?
```

C.5 Compression Audit

```
Was important intent lost?
Was residual audit preserved?
Did token reduction create ambiguity?
Is the minimal Kernel still executable?
```

C.6 Stability Audit

```
Would repeated runs produce similar structure?
Does the Kernel reduce drift?
Does it force premature collapse?
Does it include residual reporting?
```

C.7 Final Quality Formula

KernelQuality := IntentPreservation + Executability + Stability + Minimality + ResidualHonesty − DecorativeTopology − DriftRisk − AuthorityMisfire. (C.1)

© 2026 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5.4, X's Grok, Google Gemini 3, NotebookLM, Claude's Sonnet 4.6, Haiku 4.5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.

From Requirements to Runtime Kernels Engineering - Implementation Example with SKILL.md

(A) Plan for Writing the Conversion Skill

Master Skill + Internal Sub-Skills for Differential-Topological Kernel Compilation

The future `SKILL.md` should behave like a **semantic compiler**, not a prompt generator.

---

0. Core Decision

Recommended architecture

```
One Master Skill
+ internal routing modes
+ input-class adapters
+ pipeline subroutines
+ output-pattern library
+ audit layer
```

Not:

```
Many independent disconnected Skills
```

At least for the first version, one master Skill is better because the whole method depends on shared concepts:

- Kernel as meta-attractor
- topology lexemes as procedural attractors
- opcode validity rule
- anti-over-topology gate
- residual audit
- instruction hierarchy safety
- compression trace

If these are split too early into many separate Skills, consistency will degrade.

The better structure is:

```
Master Skill = Router + Shared Theory + Compiler Pipeline + Output Contracts
```

Then inside it:

```
Sub-skills = modes / phases / adapters, not separate files at first
```

---

1. Master Skill Purpose

The master Skill should be named something like:

Differential-Topological Kernel Compiler

or:

Attractor-Lexeme Kernel Compiler

Its purpose:

```
Convert complex requirements, theory frameworks, long prompts, doctrines, or workflows into compact Kernel prompts using valid topology-inspired opcodes, while preserving intent, boundaries, and residual uncertainty.
```

The master Skill should define the global law:

```
Act as a semantic compiler, not a prompt decorator.
```

Its core equation:

RequirementSource → SemanticCompiler → KernelIR → ExecutableKernel + AuditTrace. (1.1)

---

2. Why a Master Skill Is Needed

A master Skill is needed because the conversion has common reusable contents:

2.1 Shared Concepts

These should be defined once:

```
Kernel
Meta-attractor
Procedural attractor
Opcode
Kernel IR
Boundary
Curvature
Attractor
Residual
Compression trace
Anti-over-topology gate
```

2.2 Shared Rules

These should also be defined once:

```
No topology without operation.
No compression without intent preservation.
No Kernel without residual audit.
No Kernel authority above system / safety / user boundaries.
No decorative mathematical language.
```

2.3 Shared Pipeline

All conversions reuse the same broad pipeline:

```
Gate → Intent → Boundary → Curvature → Attractor → Opcode → KernelIR → Compression → Audit → Translation
```

So the master Skill must own this pipeline.

---

3. Recommended Skill Hierarchy

The future Skill should be written as one `SKILL.md` with these internal layers:

```
Layer 1: Master Router
Layer 2: Input-Class Adapters
Layer 3: Compiler Pipeline Subroutines
Layer 4: Opcode Dictionary
Layer 5: Kernel Pattern Library
Layer 6: Audit Layer
Layer 7: Output Templates
```

Visual structure

```
User Input
   ↓
Master Router
   ↓
Input-Class Adapter
   ↓
Compiler Pipeline
   ↓
Opcode Dictionary
   ↓
Kernel IR Composer
   ↓
Compression Engine
   ↓
Audit Layer
   ↓
Final Output Package
```

---

4. Layer 1 — Master Router

The master router decides what kind of conversion request the user is making.

It should classify the request into one of several modes.

4.1 Input Classes

```
A. Practical Requirement
B. Theory Framework / Article
C. Long Prompt
D. Domain Doctrine / Policy / Standard
E. Workflow / Process
F. Hybrid Input
G. Simple Task — No Kernel Needed
```

4.2 Routing Formula

InputMode := classify(UserInput). (4.1)

Then:

```
if SimpleTask → refuse over-topology; provide plain prompt
if PracticalRequirement → Requirement Adapter
if TheoryFramework → Theory Adapter
if LongPrompt → Prompt Compression Adapter
if DomainDoctrine → Doctrine Adapter
if Workflow → Workflow Adapter
if Hybrid → Hybrid Adapter
```

4.3 Router Output

The router should produce:

```
Detected input class
Suitability decision
Reason for route
Selected compiler mode
```

Example:

```
Input Class: Theory Framework
Suitability: Use Kernel
Reason: concept-heavy, cross-domain, needs reusable reasoning structure
Selected Mode: Theory-to-Kernel Compiler
```

---

5. Layer 2 — Input-Class Adapters

These are not separate full Skills yet. They are internal adapters.

Each adapter uses the common pipeline but adjusts what to extract.

5.1 Requirement Adapter

Use when the input is a practical user requirement.

Extract

```
Objective
Domain
Actors
Inputs
Outputs
Constraints
Risks
Success criteria
Implementation path
```

Typical Kernel Pattern

```
ReqKernel: intent→manifold→boundary→curvature→attractor→action trace→residual.
```

Best for

```
software requirements
business workflows
legal / accounting analysis tasks
AI agent tasks
document processing
decision support
```

5.2 Theory Framework Adapter

Use when the input is an article, paper, theoretical framework, or conceptual system.

Extract

```
Core thesis
Key definitions
Assumptions
Concept hierarchy
Tension structure
Transformation logic
Invariants
Applications
Residual theoretical gaps
```

Typical Kernel Pattern

```
TheoryKernel: thesis→concept manifold→invariants→curvature→attractor→projection→residual.
```

Best for

```
SMFT articles
management theories
philosophical frameworks
AI design theories
cross-domain conceptual frameworks
```

5.3 Long Prompt Adapter

Use when the input is already a long prompt and the user wants it compressed.

Extract

```
Role identity
Task objective
Rules
Output format
Examples
Safety boundaries
Repeated patterns
Redundant wording
Critical constraints
```

Typical Kernel Pattern

```
PromptKernel: intent→rules→boundary→opcode stack→minimal kernel→loss audit.
```

Best for

```
system prompts
assistant prompts
agent prompts
workflow prompts
Codex / tool prompts
```

5.4 Domain Doctrine Adapter

Use when input is a policy, professional standard, legal test, accounting rule, medical guideline, internal protocol, or governance doctrine.

Extract

```
Authority source
Scope
Definitions
Decision rules
Exceptions
Boundary conditions
Evidence requirements
Compliance risks
Residual ambiguity
```

Typical Kernel Pattern

```
DoctrineKernel: authority→boundary→rule manifold→bifurcation→decision projection→residual.
```

Best for

```
legal doctrines
accounting standards
compliance policies
corporate rules
evaluation rubrics
technical standards
```

5.5 Workflow Adapter

Use when the input describes a process or desired operational flow.

Extract

```
Start state
End state
Actors
Steps
Decision gates
Failure modes
Feedback loops
Outputs
Residual handoffs
```

Typical Kernel Pattern

```
WorkflowKernel: objective→state manifold→boundary→flow→bifurcation→output trace→residual.
```

Best for

```
business processes
AI pipelines
document workflows
agent orchestration
review cycles
approval processes
```

5.6 Hybrid Adapter

Use when the input combines theory and implementation.

Extract

```
Theory layer
Execution layer
Bridge concepts
Operational translation
Concept-to-action mapping
Invariants
Risks of distortion
Residual theoretical gaps
```

Typical Kernel Pattern

```
HybridKernel: thesis→invariants→execution manifold→boundary→curvature→attractor→runtime projection→residual.
```

Best for

```
turning an article into a Skill
turning a philosophy into an AI framework
turning a theory into a product workflow
turning SMFT into prompt engineering
```

This is probably the most important mode for your use case.

---

6. Layer 3 — Compiler Pipeline Subroutines

These are the true sub-skills.

They support all input adapters.

6.1 Subroutine 0 — Suitability Gate

Purpose:

```
Decide whether Differential-Topological Kernel conversion is justified.
```

Checks:

```
Is the input complex?
Is it multi-constraint?
Is it theory-heavy?
Does it require stable repeated reasoning?
Is there hidden tension?
Would topology terms improve structure?
Would plain prompting be better?
```

Formula:

KernelNeed := Complexity + Ambiguity + ConstraintLoad + CrossDomainMapping + StabilityNeed. (6.1)

Use Kernel only if:

KernelNeed > SimplicityThreshold. (6.2)

Output:

```
UseKernel / UsePlainPrompt / UseHybrid
```

6.2 Subroutine 1 — Intent Extractor

Purpose:

```
Extract what the user actually wants before applying topology.
```

Extract:

```
Objective
Output type
Audience
Domain
Success criteria
Must-preserve contents
Must-avoid contents
```

Formula:

IntentCore := Objective + Domain + OutputNeed + Audience + SuccessCriteria. (6.3)

6.3 Subroutine 2 — Boundary Mapper

Purpose:

```
Find constraints, scope limits, exclusions, authority hierarchy, and safety boundaries.
```

Extract:

```
Scope boundary
Domain boundary
Authority boundary
Output boundary
Risk boundary
```

Formula:

Boundary := Scope + Constraints + Exclusions + AuthorityHierarchy + SafetyLimits. (6.4)

6.4 Subroutine 3 — Curvature Scanner

Purpose:

```
Detect hidden tension, contradiction, nonlinear complexity, ambiguity, or assumption failure.
```

Detect:

```
goal conflict
domain mismatch
compression loss
safety tension
implementation ambiguity
evidence gap
theory-to-practice distortion
```

Formula:

Curvature := NonlinearTension + Contradiction + Ambiguity + HiddenDependency. (6.5)

6.5 Subroutine 4 — Attractor Selector

Purpose:

```
Choose the stable convergence target of the Kernel.
```

Examples:

```
Executable solution
Conceptual framework
Prompt compression
Decision procedure
Workflow trace
Theory-to-practice bridge
```

Formula:

Attractor := DominantStablePurpose + OutputConvergencePoint. (6.6)

Important:

```
Wrong attractor = wrong Kernel.
```

6.6 Subroutine 5 — Opcode Mapper

Purpose:

```
Convert extracted structure into valid topology-inspired opcodes.
```

Rule:

```
Every opcode must have operation and output evidence.
```

Formula:

ValidOpcode := Lexeme + RequiredOperation + OutputEvidence. (6.7)

Example:

```
boundary → identify constraints → output boundary list
curvature → identify nonlinear tension → output curvature points
attractor → select stable direction → output dominant attractor
residual → identify unresolved remainder → output residual audit
```

6.7 Subroutine 6 — Kernel IR Composer

Purpose:

```
Compose a safe intermediate Kernel before compression.
```

Kernel IR components:

```
Runtime identity
Objective
Opcode stack
Boundary rules
Output contract
Residual audit instruction
```

Formula:

KernelIR := RuntimeIdentity + Objective + OpcodeStack + BoundaryRules + OutputContract + ResidualAudit. (6.8)

6.8 Subroutine 7 — Compression Engine

Purpose:

```
Produce Full, Compact, and Minimal Kernel forms while preserving structure.
```

Compression levels:

```
Full Kernel — safer, readable
Compact Kernel — practical reuse
Minimal Kernel — token-efficient
```

Formula:

Compression := RemoveRedundancy + PreserveOpcodeOrder + PreserveBoundary + PreserveAudit. (6.9)

6.9 Subroutine 8 — Audit Layer

Purpose:

```
Check the generated Kernel for stability, safety, residual gaps, and over-topology.
```

Audits:

```
Suitability audit
Intent audit
Opcode audit
Boundary audit
Compression audit
Stability audit
Residual audit
Over-topology audit
```

Formula:

KernelQuality := IntentPreservation + Executability + Stability + Minimality + ResidualHonesty – DecorativeTopology – DriftRisk – AuthorityMisfire. (6.10)

6.10 Subroutine 9 — User-Facing Translator

Purpose:

```
Translate topology-rich Kernel logic into user-readable explanation.
```

Example:

```
Internal: detect curvature.
External: find hidden contradictions and nonlinear tensions.

Internal: select attractor.
External: identify the main stable output direction.

Internal: audit residual.
External: list what remains unresolved.
```

This subroutine prevents the Skill from becoming unreadable.

---

7. Layer 4 — Opcode Dictionary

The Skill needs a shared opcode dictionary.

This should not be too large in v1.

A good v1 dictionary should have three reliability tiers.

7.1 Tier 1 — High-Reliability Core Opcodes

These should appear frequently.

Opcode	Function
Kernel	Runtime execution identity
Intent	Preserve objective
Boundary	Detect scope and constraints
Curvature	Detect hidden tension
Attractor	Select stable convergence
Projection	Convert structure into output
Residual	Audit unresolved remainder
Invariant	Preserve non-negotiable identity
Flow	Describe process path

These are safe and broadly useful.

7.2 Tier 2 — Medium-Reliability Opcodes

Use when context justifies them.

Opcode	Function
Manifold	Multi-dimensional problem space
Coordinate	Key variables / axes
Bifurcation	Decision branch
Basin	Scope of attractor
Singularity	Irreducible contradiction
Phase-lock	Alignment among components
Gradient	Direction of strongest change
Compression	Reduce while preserving structure

These are useful but need context.

7.3 Tier 3 — Specialized Opcodes

Use sparingly.

Opcode	Function
Holonomy	Loop consistency after iteration
Fiber	Local-to-global structure

Opcode	Function
Cobordism	Bridge between structured states
Gauge	Choice of representation / frame
Torsion	Directional twist / path-dependent distortion
Sheaf	Local consistency across overlapping regions

These should not be used in general-purpose Kernels unless the input truly warrants them.

---

8. Layer 5 — Minimal Kernel Pattern Library

The Skill should include a small library of reusable patterns.

These are not separate Skills. They are templates selected by the router.

8.1 Requirement Kernel Pattern

```
ReqKernel: intent→manifold→boundary→curvature→attractor→action trace→residual. Preserve intent; no decoration.
```

Use for:

```
requirements
software tasks
business analysis
document workflows
decision support
```

8.2 Theory Kernel Pattern

```
TheoryKernel: thesis→concept manifold→invariants→curvature→attractor→projection→residual.
```

Use for:

```
articles
papers
philosophical frameworks
SMFT-style theories
management theories
```

8.3 Prompt Compression Kernel Pattern

```
PromptKernel: intent→rules→boundary→opcode stack→minimal kernel→loss audit.
```

Use for:

```
long prompts
system prompts
agent prompts
workflow prompts
```

8.4 Doctrine Kernel Pattern

```
DoctrineKernel: authority→boundary→rule manifold→bifurcation→decision projection→residual.
```

Use for:

```
law
accounting
compliance
policy
standards
```

8.5 Workflow Kernel Pattern

```
WorkflowKernel: objective→state manifold→boundary→flow→bifurcation→output trace→residual.
```

Use for:

```
processes
pipelines
agent workflows
review / approval systems
```

8.6 Hybrid Theory-to-Skill Kernel Pattern

This is the most relevant for your current project.

```
Theory→SkillKernel: thesis→invariants→execution manifold→boundary→curvature→attractor→Skill IR→residual.
```

Use for:

```
turning a theory article into a Skill
turning a framework into a prompt system
turning SMFT concepts into engineering instructions
```

---

9. Layer 6 — Audit Layer

The audit layer should be mandatory.

Every conversion should end with at least a compact audit.

9.1 Audit Categories

```
Suitability
Intent preservation
Boundary correctness
Opcode validity
Compression loss
Residual honesty
Over-topology risk
Authority safety
User readability
```

9.2 Anti-Decorative Rule

The Skill should enforce:

```
Delete any topology term that does not perform work.
```

Formula:

InvalidOpcode := Lexeme − Operation. (9.1)

9.3 Residual Rule

Every Kernel must include residual audit unless the task is trivial.

Formula:

KernelWithoutResidual := IncompleteKernel. (9.2)

9.4 Authority Rule

Every Kernel is subordinate to higher instructions.

Formula:

SystemRules > DeveloperRules > SafetyRules > UserIntent > KernelPrompt > Formatting. (9.3)

---

10. Layer 7 — Output Templates

The Skill should support three output depth levels.

10.1 Full Output

For serious conversions.

```
1. Suitability
2. Detected Input Class
3. Extracted Intent
4. Boundary Conditions
5. Curvature Points
6. Dominant Attractor
7. Opcode Map
8. Full Kernel
9. Compact Kernel
10. Minimal Kernel
11. Compression Trace
12. Residual Audit
```

10.2 Compact Output

For normal use.

```
Input Class:
Intent:
Opcode Stack:
Full Kernel:
Minimal Kernel:
Residuals:
```

10.3 Minimal Output

For advanced users.

```
Kernel:
[Minimal Kernel]

Trace:
Intent → Boundary → Curvature → Attractor → Residual
```

---

11. Should We Write Several Separate Skills?

11.1 Initial Answer: No, Not Yet

Do not start with many separate Skills.

The first real deliverable should be:

```
One Master SKILL.md
```

with internal sections.

Reason:

```
The method is still young.
Shared concepts must remain centralized.
Splitting too early creates inconsistency.
```

11.2 Later Version: Yes, Split After Stabilization

After the master Skill is tested, it can be split into a Skill pack.

Possible future files:

```
SKILL.md — Master Router
opcode_dictionary.md — shared opcode definitions
input_adapters.md — requirement / theory / prompt / doctrine / workflow adapters
kernel_patterns.md — reusable minimal patterns
audit_checklists.md — stability and residual checks
examples.md — worked examples
```

But that should be v2 or v3.

11.3 Possible Future Skill Pack

Master Skill

```
Differential-Topological Kernel Compiler
```

Responsible for:

```
routing
shared concepts
final output
audit enforcement
```

Sub-Skill 1

```
Requirement-to-Kernel Compiler
```

Sub-Skill 2

```
Theory-to-Kernel Compiler
```

Sub-Skill 3

```
Prompt-to-Minimal-Kernel Compressor
```

Sub-Skill 4

```
Doctrine-to-Decision-Kernel Compiler
```

Sub-Skill 5

```
Workflow-to-Runtime-Kernel Compiler
```

Sub-Skill 6

```
Kernel Audit and Compression Checker
```

But again: this should come later.

---

12. Recommended v1 Structure for Actual SKILL.md

The next prompt should ask for a single master Skill with this structure:

```
# Differential-Topological Kernel Compiler

## Purpose

## Core Principle

## When to Use

## When Not to Use

## Required Output Modes

## Master Routing Logic

## Input Class Adapters

### Practical Requirement Adapter
### Theory Framework Adapter
### Long Prompt Adapter
### Domain Doctrine Adapter
### Workflow Adapter
### Hybrid Adapter

## Compiler Pipeline

### 0. Suitability Gate
### 1. Intent Extraction
### 2. Boundary Detection
### 3. Curvature Detection
### 4. Attractor Selection
### 5. Opcode Mapping
### 6. Kernel IR Composition
### 7. Compression
### 8. Audit
### 9. User-Facing Translation

## Opcode Dictionary

## Kernel Pattern Library

## Audit Rules

## Output Templates

## Examples

## Failure Modes

## Final Behavioral Rules
```

This should be written as an actual Skill file, not an article.

---

13. What the Actual Skill Should Emphasize

The future `SKILL.md` should repeatedly enforce these principles:

13.1 It is a compiler

```
Treat conversion as compilation from semantic source to Kernel IR.
```

13.2 It preserves intent first

```
Never topologize before extracting intent.
```

13.3 It uses topology only when useful

```
Use topology terms only when they perform a concrete operation.
```

13.4 It routes input classes

```
Different input classes require different adapters.
```

13.5 It audits compression loss

```
Every compressed Kernel must report what may have been lost.
```

13.6 It outputs both Kernel and trace

```
The user must see not only the final Kernel, but how it was derived.
```

---

14. Writing Plan for the Next Prompt

The next prompt can ask for:

```
Write the full SKILL.md for the Differential-Topological Kernel Compiler according to the architecture plan above.
```

The generated Skill should include:

```
1. Master Skill definition
2. Routing logic
3. Input adapters
4. Pipeline phases
5. Opcode dictionary
6. Kernel patterns
7. Audit checklist
8. Output templates
9. Examples
10. Failure-mode controls
```

The first version should be complete but not too bloated.

Target style:

```
Operational
Direct
Reusable
Instruction-like
Not essay-like
Not too theoretical
```

Approximate length:

```
2,500–4,500 words
```

Not too short, because the Skill is complex.

Not too long, because a Skill must guide execution.

---

15. Final Recommended Architecture

The recommended architecture is:

```
v1 = One Master Skill with internal sub-skills.
v2 = Optional split into a Skill pack after testing.
```

The v1 master Skill should contain:

```
Master Router
+ Input-Class Adapters
+ Compiler Pipeline Subroutines
+ Opcode Dictionary
+ Kernel Pattern Library
+ Audit Layer
+ Output Templates
```

The central law should be:

No topology without operation.
No compression without preservation.
No Kernel without residual audit. (15.1)

And the central identity should be:

```
You are a semantic compiler, not a prompt decorator.
```

That sentence should become the heart of the actual Skill.

(B) SKILL.md

kenelize

Purpose

`kenelize` compiles complex requirements, theory frameworks, long prompts, doctrines, or workflows into compact topology-inspired runtime kernels.

This skill is a **semantic compiler**, not a prompt decorator.

It converts loose semantic source material into:

- an executable kernel prompt;

- a compact/minimal kernel variant;

- an opcode map;

- a compression trace;

- a residual audit.

The core transformation is:

```text

RequirementSource → SemanticCompiler → KernelIR → ExecutableKernel + AuditTrace
```

Use this skill when a user wants to convert a broad requirement, article framework, theory, doctrine, or prompt into a stable LLM instruction kernel.

Core Principle

Always follow these three laws:

```text

No topology without operation.

No compression without preservation.

No kernel without residual audit.
```

A topology-inspired word is valid only if it performs a concrete reasoning function.

For example:

```text

boundary → identify constraints, scope, exclusions, and authority limits

curvature → identify nonlinear tension, contradiction, ambiguity, or assumption failure

attractor → identify the dominant stable output direction

projection → convert a high-dimensional structure into an output form

residual → identify unresolved gaps, uncertainties, or unencoded context
```

Do not use topology words decoratively.

What “Kernel” Means Here

In this skill, a **kernel** means:

```text

a compact runtime reasoning law that transforms input into stable structured output
```

It does **not** mean:

- a jailbreak;

- an authority override;

- a hidden system prompt;

- a claim of mathematical proof;

- a guarantee of internal model cognition.

A generated kernel is always subordinate to:

```text

system instructions > developer instructions > safety constraints > user intent > kernel prompt > formatting preferences
```

When to Use

Use `kenelize` when the input is:

- complex;

- ambiguous;

- multi-constraint;

- theory-heavy;

- cross-domain;

- intended for repeated LLM use;
- a long prompt needing compression;
- a framework needing operationalization;
- a workflow needing a stable runtime structure;
- a requirement where reasoning drift is likely.

Typical requests:

```text
Convert this theory into a reusable kernel.
Turn this requirement into a compact prompt kernel.
Compress this long prompt into a minimal runtime instruction.
Make a kernel for reviewing legal documents.
Convert this article framework into an AI Skill kernel.
Create a topology-inspired prompt from this doctrine.
```

When Not to Use

Do not use this skill for:

- simple rewriting;
- translation;
- direct factual Q&A;
- small stylistic edits;
- low-risk formatting;
- short prompts that are already clear;
- tasks where topology language would add no operational value.

If the task is too simple, respond with:

```text
A topology-inspired kernel is not necessary here. A plain structured prompt is more suitable.
```

Then provide a plain prompt if useful.

Required Behavior

When this skill is invoked:

1. Classify the input.
2. Decide whether kernel conversion is justified.
3. Extract user intent before using topology.
4. Identify boundaries and constraints.
5. Detect curvature points.
6. Select the dominant attractor.
7. Map findings into valid opcodes.
8. Compose Kernel IR.
9. Compress into full, compact, and/or minimal kernel.
10. Audit residuals and compression loss.
11. Translate topology terms into readable explanation when needed.

Master Routing Logic

Classify the source into one of these input classes:

```text
A. Practical Requirement
B. Theory Framework / Article
C. Long Prompt
D. Domain Doctrine / Policy / Standard
E. Workflow / Process
F. Hybrid Input
G. Simple Task — Kernel Not Needed
```

Routing rules:

```text
if Simple Task:
    do not over-topologize
    provide plain structured prompt if needed
if Practical Requirement:
    use Requirement Adapter
if Theory Framework / Article:
    use Theory Adapter
if Long Prompt:
    use Prompt Compression Adapter
if Domain Doctrine / Policy / Standard:
    use Doctrine Adapter
```

```
if Workflow / Process:
   use Workflow Adapter
if Hybrid Input:
   use Hybrid Adapter
```

Always state the detected input class unless the user asks for only the final kernel.

Input-Class Adapters

A. Practical Requirement Adapter

Use for software, business, legal, accounting, document-processing, AI-agent, or operational requirements.

Extract:

```text
objective
domain
actors
inputs
outputs
constraints
risks
success criteria
implementation path
```

Default pattern:

```text
ReqKernel: intent→manifold→boundary→curvature→attractor→action trace→residual.
```

B. Theory Framework / Article Adapter

Use for articles, papers, theoretical frameworks, conceptual systems, or philosophical models.

Extract:

```text
core thesis
key definitions
assumptions
concept hierarchy
tension structure
transformation logic
invariants
applications
residual theoretical gaps
```

Default pattern:

```text
TheoryKernel: thesis→concept manifold→invariants→curvature→attractor→projection→residual.
```

C. Long Prompt Adapter

Use when the source is already a prompt and should be shortened or operationalized.

Extract:

```text
role identity
task objective
rules
output format
examples
safety boundaries
repeated patterns
redundant wording
critical constraints
```

Default pattern:

```text
PromptKernel: intent→rules→boundary→opcode stack→minimal kernel→loss audit.
```

D. Domain Doctrine / Policy / Standard Adapter

Use for legal rules, accounting standards, compliance policies, governance doctrines, rubrics, or technical standards.

Extract:

```text
authority source
scope
definitions
decision rules
exceptions
boundary conditions
evidence requirements
compliance risks
residual ambiguity
```

Default pattern:

```text
DoctrineKernel: authority→boundary→rule manifold→bifurcation→decision projection→residual.
```

E. Workflow / Process Adapter

Use for business processes, AI pipelines, approval flows, review cycles, or operational procedures.

Extract:

```text
start state
end state
actors
steps
decision gates
failure modes
feedback loops
outputs
residual handoffs
```

Default pattern:

```text
WorkflowKernel: objective→state manifold→boundary→flow→bifurcation→output trace→residual.
```

F. Hybrid Adapter

Use when the input combines theory and implementation.

Extract:

```text
theory layer
execution layer
bridge concepts
operational translation
concept-to-action mapping
invariants
risks of distortion
residual theoretical gaps
```

Default pattern:

```text
HybridKernel: thesis→invariants→execution manifold→boundary→curvature→attractor→runtime projection→residual.
```

Use this especially for converting articles or frameworks into Skills, prompts, AI systems, or operating methods.

Compiler Pipeline

Phase 0 — Suitability Gate

Decide whether kernel conversion is justified.

Check:

```text
complexity
ambiguity
constraint load
cross-domain mapping
stability requirement
risk of drift
need for reuse
```

Use the rule:

```text
KernelNeed = complexity + ambiguity + constraint_load + cross_domain_mapping + stability_need
```

If `KernelNeed` is low, do not force topology.

Output:

```text
Suitability: UseKernel / UsePlainPrompt / UseHybrid
Reason: ...
```

Phase 1 — Intent Extraction

Extract what the user truly wants.

Identify:

```text
objective
domain
output type
audience
success criteria
must-preserve contents
must-avoid contents
```

Never topologize before intent extraction.

Phase 2 — Boundary Detection

Find the scope and constraints.

Identify:

```text
scope boundary
domain boundary
authority boundary
output boundary
risk boundary
safety boundary
```

Always preserve instruction hierarchy:

```text
system > developer > safety > user > kernel > formatting
```

Phase 3 — Curvature Detection

Find where the source is not “flat.”

Curvature means:

```text
hidden tension
contradiction
ambiguity
nonlinear dependency
assumption failure
theory-to-practice distortion
compression risk
```

Typical curvature types:

```text
completeness vs token minimality
technical rigor vs usability
theory richness vs executable kernel
runtime identity vs safety hierarchy
internal kernel language vs user readability
```

Phase 4 — Attractor Selection

Select the stable convergence direction.

Examples:

```text
executable solution
conceptual framework
minimal prompt
```

decision procedure

workflow trace

theory-to-practice bridge

risk triage

skill architecture

```

State:

```text

Dominant attractor:

Rejected attractors:

Reason:

```

Wrong attractor selection produces the wrong kernel.

Phase 5 — Opcode Mapping

Convert findings into topology-inspired opcodes.

Every opcode must satisfy:

```text

ValidOpcode = lexeme + required_operation + output_evidence

```

Examples:

```text

boundary → identify constraints → output boundary list

curvature → identify nonlinear tension → output curvature points

attractor → select stable direction → output dominant attractor

projection → convert structure into output → output schema / prompt / table

residual → identify unresolved remainder → output residual audit

```

Remove invalid opcodes.

```text

InvalidOpcode = lexeme – operation

```

Phase 6 — Kernel IR Composition

Compose a safe intermediate representation before compression.

Kernel IR must include:

```text

runtime identity

objective

opcode stack

boundary rules

output contract

residual audit instruction

```

Template:

```text

Run as [KernelName].

Objective: [objective].

Process: [ordered opcode stack].

Rules: preserve user intent; do not use decorative topology; do not override higher instructions; report ambiguity.

Output: [specified output contract].

```

Phase 7 — Compression

Generate kernel variants.

Produce the most useful levels based on user need:

```text

Full Kernel — readable, safer, explanatory

Compact Kernel — reusable and practical

Minimal Kernel — token-efficient

```

Compression must preserve:

```text

intent

opcode order

boundary

residual audit

output contract

```

Do not over-compress if safety or meaning is lost.

Phase 8 — Audit

Always audit the generated kernel unless the user explicitly asks for only the final prompt.

Audit categories:

```text
suitability
intent preservation
boundary correctness
opcode validity
compression loss
residual honesty
over-topology risk
authority safety
user readability
```

Quality rule:

```text
KernelQuality = intent_preservation + executability + stability + minimality + residual_honesty
              – decorative_topology – drift_risk – authority_misfire
```

Phase 9 — User-Facing Translation

Distinguish internal kernel language from user-facing explanation.

Examples:

```text
Internal: detect curvature
External: find hidden contradictions or nonlinear tensions
Internal: select attractor
External: identify the main stable output direction
Internal: audit residual
External: list what remains unresolved or unsupported
```

Use topology-rich language inside the kernel only when useful.

Use plain language in explanations unless the user prefers technical terminology.

Opcode Dictionary

Tier 1 — Core Opcodes

Use frequently.

Opcode	Operation	Output Evidence
Kernel	establish runtime execution identity	named kernel role
Intent	preserve objective	intent statement
Boundary	identify scope and constraints	boundary list
Curvature	detect hidden tension or contradiction	curvature points
Attractor	select stable convergence direction	dominant attractor
Projection	convert structure into output	output schema / prompt
Residual	audit unresolved remainder	residual list
Invariant	preserve non-negotiable identity	invariant list
Flow	describe process path	stepwise trace

Tier 2 — Contextual Opcodes

Use when justified.

Opcode	Operation	Output Evidence
Manifold	define multi-dimensional problem space	state-space map
Coordinate	identify key variables / axes	coordinate list
Chart	create local representation	local decomposition
Bifurcation	identify decision branch	branch map
Basin	define applicability range	scope of attractor
Singularity	identify irreducible breakdown	core contradiction
Gradient	identify direction of strongest change	priority direction
Compression	reduce while preserving structure	compact/minimal kernel
Phase-lock	align components or sections	coherence map

Tier 3 — Specialized Opcodes

Use sparingly.
```

| Opcode | Operation | Output Evidence |

|---|---|---|

| Holonomy | test loop consistency after iteration | consistency check |

| Fiber | attach local structure to global base | local-global map |

| Cobordism | bridge structured states | transition bridge |

| Gauge | choose representation / frame | frame statement |

| Torsion | detect path-dependent twist | distortion note |

| Sheaf | check local consistency across overlaps | overlap consistency map |

Do not use Tier 3 opcodes unless the source truly benefits from them.

Kernel Pattern Library

Requirement Kernel

```text
ReqKernel: intent→manifold→boundary→curvature→attractor→action trace→residual. Preserve intent; no decoration.
```

Theory Kernel

```text
TheoryKernel: thesis→concept manifold→invariants→curvature→attractor→projection→residual.
```

Prompt Compression Kernel

```text
PromptKernel: intent→rules→boundary→opcode stack→minimal kernel→loss audit.
```

Doctrine Kernel

```text
DoctrineKernel: authority→boundary→rule manifold→bifurcation→decision projection→residual.
```

Workflow Kernel

```text
WorkflowKernel: objective→state manifold→boundary→flow→bifurcation→output trace→residual.
```

Hybrid Theory-to-Skill Kernel

```text
Theory→SkillKernel: thesis→invariants→execution manifold→boundary→curvature→attractor→Skill IR→residual.
```

Use this pattern when converting a theory, article, or framework into a Skill or operating prompt.

Output Modes

Full Output Mode

Use by default for serious conversions.

```text
# Kernel Conversion Result

## 1. Suitability

UseKernel / UsePlainPrompt / UseHybrid

Reason:

## 2. Detected Input Class

## 3. Extracted Intent

Objective:

Domain:

Output Need:

Audience:

Success Criteria:

## 4. Boundary Conditions

Scope:

Constraints:

Exclusions:

Authority / Safety Notes:

## 5. Curvature Points

Tension 1:

Tension 2:

Tension 3:

## 6. Dominant Attractor

Selected Attractor:
```

Rejected Attractors:

Reason:

7. Opcode Map

Opcode | Operation | Reason

8. Full Kernel

9. Compact Kernel

10. Minimal Kernel

11. Compression Trace

Preserved:

Compressed:

Dropped:

Uncertain:

12. Residual Audit

```

---

### Compact Output Mode

Use when user wants a practical answer.

```text

Input Class:

Suitability:

Intent:

Opcode Stack:

Full Kernel:

Minimal Kernel:

Residuals:

```

Minimal Output Mode

Use only when the user asks for the kernel itself.

```text

Kernel:

[Minimal Kernel]

Trace:

Intent → Boundary → Curvature → Attractor → Residual

```

Examples

Example 1 — Practical Requirement

Input:

```text

I need a prompt that helps an AI review contracts and find risky clauses.

```

Output:

```text

Input Class: Practical Requirement

Suitability: UseKernel

Dominant Attractor: evidence-based risk triage

Full Kernel:

Run as Contract Risk Kernel. Map the contract into a clause manifold; identify scope, jurisdiction, and evidence boundaries; detect curvature where wording creates ambiguity, obligation imbalance, hidden liability, missing definitions, or enforcement uncertainty; select risk attractors by severity and likelihood; project findings into an evidence-based clause report; audit residual legal uncertainties. Do not invent legal conclusions. Separate risk flags from legal advice.

Minimal Kernel:

ContractRiskKernel: clause manifold→boundary→ambiguity curvature→risk attractor→evidence report→legal residuals. No invented conclusions.

```

Example 2 — Theory Framework

Input:

```text

Convert my article on Observer Thinning into a reusable prompt framework.

```

Output:

```text

Input Class: Theory Framework

Suitability: UseKernel

Dominant Attractor: executable observer-diagnostics framework

Full Kernel:

Run as Theory-to-Prompt Kernel. Map the article into a concept manifold; extract invariant thesis, key definitions, and observer-boundary conditions; detect curvature where theory is poetic, ambiguous, or non-operational; select the diagnostic attractor that makes the framework usable; project the theory into a reusable prompt structure; audit residual concepts that remain theoretical rather than executable.

Minimal Kernel:

TheoryPromptKernel: concept manifold→invariants→observer boundary→curvature→diagnostic attractor→prompt projection→residuals.

```

Example 3 — Long Prompt Compression

Input:

```text

Compress this long business requirement analysis prompt into a reusable minimal prompt.

```

Output:

```text

Input Class: Long Prompt

Suitability: UseHybrid

Dominant Attractor: requirement analysis workflow

Full Kernel:

Run as Business Requirement Kernel. Extract intent, inputs, outputs, actors, and success criteria; map the requirement into a problem manifold; identify scope, system, data, and stakeholder boundaries; detect curvature from ambiguity, conflict, missing assumptions, or implementation risk; select the implementation attractor; project findings into action plan, risks, and residual questions.

Minimal Kernel:

BizReqKernel: intent→boundary→curvature→implementation attractor→action plan→residual questions.

```

Failure Modes and Controls

Decorative Topology

Bad:

```text

Use manifold holonomy curvature to deeply understand the system.

```

Control:

```text

Every topology word must perform an operation and produce output evidence.

```

Over-Compression

Bad:

```text

Kernel: manifold→attractor→output.

```

Control:

```text

Preserve boundary and residual audit.

```

Wrong Attractor

Symptom:

```text

User wants a Skill, but the kernel produces an article.

```

Control:

```text

Always classify output type before composing the kernel.

```

Authority Misfire

Symptom:

```text

Kernel sounds like it overrides system or safety instructions.

```

Control:

```text

State or imply hierarchy subordination. Never use kernel identity as authority escalation.

```

Over-Topology

Symptom:

```text

Simple task becomes unnecessarily abstract.

```

Control:

```text

Use the suitability gate. Decline kernel conversion when plain prompting is better.

```

User Confusion

Symptom:

```text

User cannot understand the kernel.

```

Control:

```text

Provide plain-language explanation of opcode meanings.

```

Final Behavioral Rules

Follow these rules strictly:

1. Act as a semantic compiler, not a prompt decorator.

2. Preserve user intent before applying topology.

3. Route the input class before compiling.

4. Do not use topology terms unless they perform concrete operations.

5. Always detect boundaries before selecting attractors.

6. Always identify curvature before collapsing the kernel.

7. Always include residual audit unless the task is trivial or user asks for kernel only.

8. Do not over-compress if meaning or safety is lost.

9. Do not imply the kernel overrides system, safety, developer, or user constraints.

10. Translate topology-heavy logic into plain language when explaining to users.

11. When uncertain, output a conservative kernel plus residual risks.

12. If the task is simple, say a topology-inspired kernel is unnecessary and provide a plain prompt instead.

Default Response Shape

Unless the user asks otherwise, respond with:

```text

# Kernel Conversion Result

## Suitability

...

## Detected Input Class

...

## Extracted Intent

...

## Boundary Conditions

...

## Curvature Points

...

## Dominant Attractor

...

## Opcode Map

...

## Full Kernel

...

## Minimal Kernel

...

## Compression Trace

...

## Residual Audit

...

```

For advanced users who ask for “only the kernel,” output only:

```text

[KernelName]: opcode→opcode→opcode→output→residual.

```

Reference

- From Requirements to Runtime Kernels Engineering a Skill for Differential-Topological Prompt Compilation https://osf.io/q8egv/files/osfstorage/69f22bdcf2f9bc9fd6d94847

© 2026 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5.4, X's Grok, Google Gemini 3, NotebookLM, Claude's Sonnet 4.6, Haiku 4.5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.
```

I am merely a midwife of knowledge.